Flathub Submission — Final Steps

Everything needed to open the PR once the local `flatpak-builder` build is green. Assumes you're submitting from a personal GitHub account with collaborator access to `github.com/besoeasy/gupt`.

---

0. Pre-flight state

Check	Expected
Tag pushed	`v0.2.0` on `origin/main`
Manifest pins SHA	`aaf5a1f4ca043c0a09dc38e483bd580e106bfd11`
Local prod build	`flatpak-builder --user --install --force-clean build-dir flatpak/com.besoeasy.gupt.yaml` → clean
App launches under Flatpak	`flatpak run com.besoeasy.gupt` → works
`appstreamcli validate`	passes

If any of the above fails, **fix before submitting**. Flathub reviewers will not iterate on a broken build.

---

1. Record a demo video (required, not skippable)

Flathub's PR template has a **mandatory video slot** enforced by both human reviewers and a GitHub Actions bot that auto-blocks incomplete submissions. Two recent attempts to mark it `N/A` (PR #8370, #8414) were publicly rejected by reviewers with comments like *"Why do you think this is in the PR template if we would not require it?"* and *"this is not an option"*. Both PRs remain blocked.

PROF

GitHub hosts the video directly when you drag the file into the comment box — no third-party account needed.

Record 20–60 seconds showing:

1. App launched from the desktop launcher (`.desktop` file working)
2. Chat UI, typing, sending a message
3. Optionally: a notification firing

Tools on Zorin / GNOME:

```
# Built-in recorder
Ctrl+Alt+Shift+R    # start — recording indicator appears
Ctrl+Alt+Shift+R    # stop — saves to ~/Videos/
```

Constraints:

- File size < 10 MB (GitHub's soft limit for drag-uploads)
- `.webm` preferred (smallest); `.mp4` fine too
- No audio required

Trim / compress if needed:

```
ffmpeg -i input.webm -c:v libvpx-vp9 -crf 35 -b:v 0 -an -vf
scale=1280:-2 output.webm
```

---

2. Fork Flathub and create the submission branch

One-time setup. Uses the GitHub CLI with your personal account (not the `besoeasy` org).

```
gh auth status                                    # confirm the active
user is YOU
gh repo fork flathub/flathub --clone=true --remote=true
cd flathub                                        # into the new clone
git checkout -b com.besoeasy.gupt                 # branch MUST match
the app-id
```

After this, `git remote -v` should show:

```
origin    git@github.com:<your-username>/flathub.git   (fetch / push)
upstream  git@github.com:flathub/flathub.git           (fetch / push)
```

PROF

If the branch already exists from a prior attempt, switch to it:

```
git checkout com.besoeasy.gupt
```

---

3. Copy the two submission files into the fork

Flathub expects only the manifest and the offline sources file at the root of the branch. The git source inside the manifest pulls everything else (launcher, desktop file, metainfo, icons, screenshots) from the tagged upstream repo.

From inside the Flathub fork directory, copy the two files from your local GUPT clone's `flatpak/` folder:

```
cp <path-to-gupt-clone>/flatpak/com.besoeasy.gupt.yaml ./
cp <path-to-gupt-clone>/flatpak/generated-sources.json ./
```

Do **not** copy `gupt.sh`, `.desktop`, `metainfo.xml`, or `screenshots/` — those live in the upstream repo at the pinned tag and will be fetched automatically during the Flathub build.

Smoke-test the production build from this fork before submitting:

```
flatpak-builder --user --install --force-clean build-dir
com.besoeasy.gupt.yaml
flatpak run com.besoeasy.gupt
```

This is the exact build Flathub's CI will run. If it fails here, fix the upstream repo, retag, update the manifest's `commit:` SHA, copy again, retry.

---

4. Commit and push submission files

```
git status          # confirm ONLY com.besoeasy.gupt.yaml + generated-
sources.json untracked
git add com.besoeasy.gupt.yaml generated-sources.json
git commit -m "Add com.besoeasy.gupt"
git push origin com.besoeasy.gupt
```

Do **not** stage `.flatpak-builder/` or `build-dir/` — these are local caches from the smoke test.

---

5. Open the PR

PROF

```
gh pr create --repo flathub/flathub --base new-pr --title "Add
com.besoeasy.gupt" --web
```

`--web` opens the composer in your browser so the template auto-loads and you can drag the video into the body.

Base branch must be `new-pr`, not `master`. The flag above enforces it.

---

6. Fill the PR body

Replace the template's checklist with this (all boxes `[X]`, drag video into the slot marked below):

```
```

```
### Please confirm your submission meets all the criteria

- [X] Please describe the application briefly. GUPT is an anonymous end-
to-end encrypted chat client built on Nostr relays, with direct
messages, WebRTC calls, encrypted media, and local-first group state.
MIT licensed, zero servers, zero accounts.
- [X] Please attach a video showcasing the application on Linux using
the Flatpak.

<!-- DRAG YOUR SCREEN RECORDING INTO THIS LINE — GitHub uploads and
inserts the URL automatically -->

- [X] The Flatpak ID follows all the rules listed in the [Application ID
requirements][appid].
- [X] I have read and followed all the [Submission requirements][reqs]
and the [Submission guide][reqs2] and I agree to them.
- [X] I am an upstream contributor to the project. I'm a collaborator on
https://github.com/besoeasy/gupt submitting on behalf of the project.

---

**Upstream:** https://github.com/besoeasy/gupt
**Release:** v0.2.0 (`aaf5a1f4ca043c0a09dc38e483bd580e106bfd11`)
**License:** MIT
**Runtime:** `org.freedesktop.Platform//25.08` +
`Electron2.BaseApp//25.08`

**Local test:** `flatpak-builder` from the production manifest clean,
`appstreamcli validate` passes, app launches, connects to relays,
notifications deliver.

**Auto-update:** Disabled inside Flatpak via `FLATPAK_ID` guard — system
update path is authoritative.

[appid]: https://docs.flathub.org/docs/for-app-
authors/requirements#application-id
[reqs]: https://docs.flathub.org/docs/for-app-authors/requirements
[reqs2]: https://docs.flathub.org/docs/for-app-authors/submission
```

PROF

7. After submission — what to expect

1. **Automated CI** runs Flathub's `flatpak-builder` on their infra. Status visible in PR checks. 5–15 min for a first build.
2. **Manual review** by a volunteer maintainer. Response window: 3–14 days. Silence ≠ rejection.
3. **Common reviewer requests:**
   - Trim `finish-args` permissions (e.g., drop `--filesystem=xdg-download` if unused)
   - Pin exact `commit:` alongside `tag:` (you already did)
   - Fix metainfo formatting / OARS rating specifics

- Add `x-checker-data` for automated dep update detection

4. **On-demand rebuild:** comment `bot, build` on the PR to retrigger CI.
5. **Iterate on the same branch** — push to `com.besoeasy.gupt` on your fork, CI rebuilds automatically.

---

8. After approval

1. Flathub admin creates `github.com/flathub/com.besoeasy.gupt`.
2. You receive a GitHub invite as maintainer — **accept within 7 days** or the invite expires.
3. Subsequent releases go to that repo (not back to `flathub/flathub`):

```
gh repo clone flathub/com.besoeasy.gupt
cd com.besoeasy.gupt
# bump manifest tag/commit, regenerate generated-sources.json, bump
metainfo release
git commit -am "Update to v0.3.0"
git push                    # master branch → beta channel
# after testing on beta:
git push origin master:stable   # promote to stable
```

4. App listing appears on flathub.org and in Software Centers within ~30 min of the first successful build.

---

9. Release workflow (future versions)

For each GUPT release:

Step 1 — in the GUPT upstream repo clone:

PROF

```
# bump upstream version, tag, push
git tag v0.3.0
git push origin main --tags

# regenerate offline sources (lockfile may have changed)
npm install --package-lock-only
flatpak-node-generator npm package-lock.json -o flatpak/generated-
sources.json

# update manifest + metainfo + push
#   - flatpak/com.besoeasy.gupt.yaml         → bump tag + commit SHA
#   - flatpak/com.besoeasy.gupt.metainfo.xml → add <release
version="0.3.0" ...>
git commit -am "Bump Flathub manifest to v0.3.0"
git push
```

Step 2 — in the Flathub dedicated repo clone (`github.com/flathub/com.besoeasy.gupt`):

```
# Copy the three files from the upstream repo's flatpak/ folder into
this repo:
#   - com.besoeasy.gupt.yaml
#   - generated-sources.json
#   - com.besoeasy.gupt.metainfo.xml (if you keep it mirrored here)
git commit -am "Release v0.3.0"
git push
```

Automate step 2 eventually with Flathub's `flatpak-external-data-checker` via an `x-checker-data` block on the git source.

---

10. Gotchas to re-check before each PR update

- Tag + commit SHA in manifest must match reality on GitHub (`git ls-remote origin v0.X.0`)
- `generated-sources.json` must be regenerated if `package-lock.json` changed
- Metainfo `<release>` block dates in `YYYY-MM-DD` format
- Screenshot URLs in metainfo still resolve over HTTPS
- No leftover `REPLACE_WITH_COMMIT_SHA` or other placeholders

---

11. Reference

- Submission docs: https://docs.flathub.org/docs/for-app-authors/submission
- Requirements: https://docs.flathub.org/docs/for-app-authors/requirements
- App ID rules: https://docs.flathub.org/docs/for-app-authors/requirements#application-id
- Electron on Flathub wiki: https://github.com/flathub/flathub/wiki/App-Requirements#electron-apps
- `flatpak-node-generator`: https://github.com/flatpak/flatpak-builder-tools/tree/master/node